UNIVERSIDADE KIMPA VITA
ESCOLAR SUPERIOR POLITÉCNICO DO UÍGE
CURSO DE ENGENHARIA INFORMÁTICA

TEMA: Relatório de Analise de Redes Sociais

(Frequência de Padrão de Conteúdo | Post Simulado)

AGENTES DO CAMPOS:

Grupo: NAC Solutions
Classe: 3º ano
Turma: Única
Período: Manhã
Data: Abril de 2026
Local: Uige, Angola

ARNANO QUIESSE MUANZA MAVINGA
CLEMENTE MANUEL DINIS JOSÉ
Augusto Paulo
JOÃONKEMBO
NSUMBO ATÓNIO NGOMBO

Orientador: ENGº. Moyo Kanivengidio

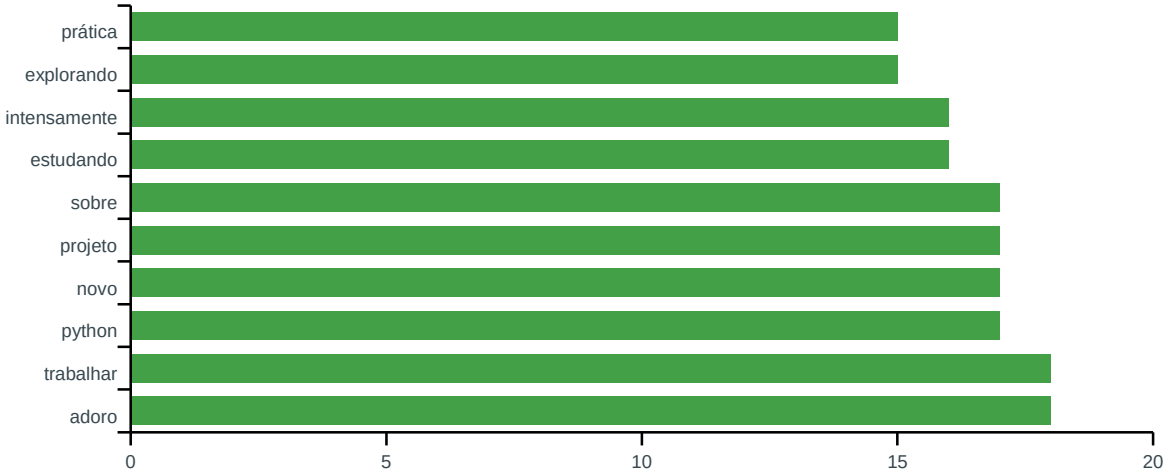
Resumo Geral

Ocorrências de Palavras	Usos de Hashtags	Menções de Utilizadores	URLs Encontradas
166	67	69	80

1. Top 10 Palavras Mais Frequentes

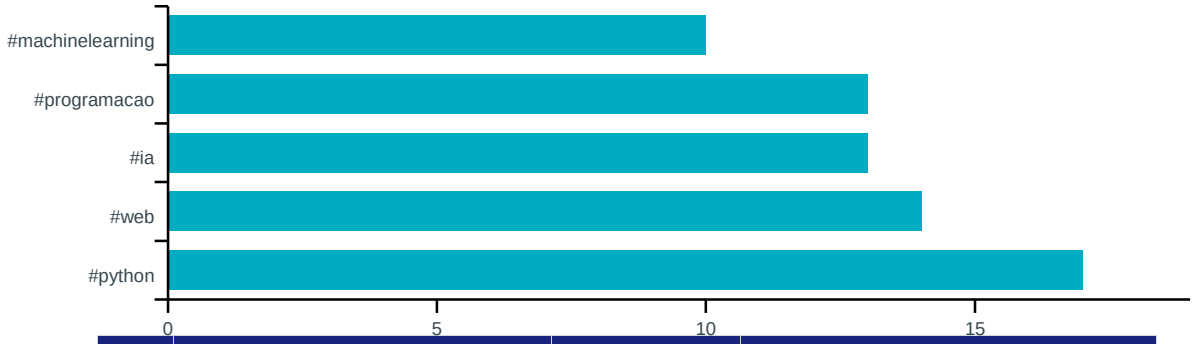
Palavras com maior ocorrência nos posts, após remoção de stopwords e tokens não-alfabéticos.

#	Palavra	Frequência	Proporção
1	adoro	18x	■■ 10.8%
2	trabalhar	18x	■■ 10.8%
3	python	17x	■■ 10.2%
4	novo	17x	■■ 10.2%
5	projeto	17x	■■ 10.2%
6	sobre	17x	■■ 10.2%
7	estudando	16x	■ 9.6%
8	intensamente	16x	■ 9.6%
9	explorando	15x	■ 9.0%
10	prática	15x	■ 9.0%



2. Top 5 Hashtags Mais Populares

Hashtags identificadas por expressão regular (#\w+), normalizadas para minúsculas.

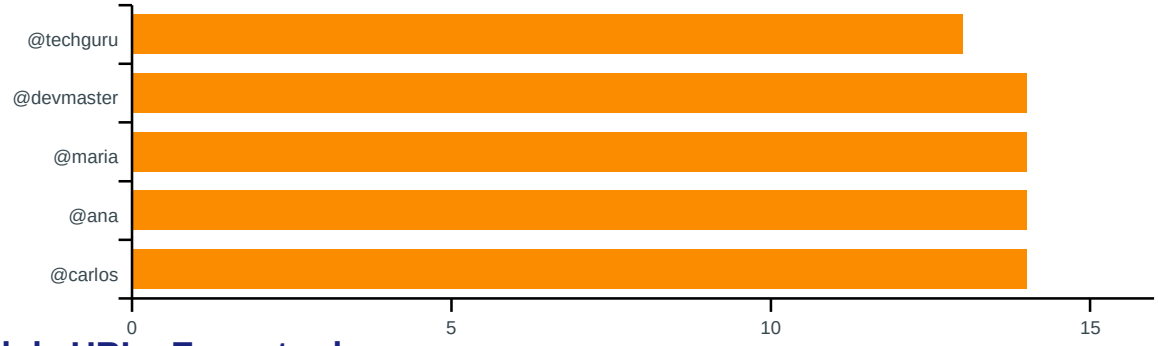


#	Hashtag	Frequência	Proporção
1	#python	**17x**	■■■■■ 25.4%
2	#web	**14x**	■■■■ 20.9%
3	#ia	**13x**	■■■ 19.4%
4	#programacao	**13x**	■■■ 19.4%
5	#machinelearning	**10x**	■■ 14.9%

3. Top 5 Menções Mais Frequentes

Utilizadores mencionados via @utilizador nos posts analisados.

#	Mencao	Frequência	Proporção
1	@carlos	**14x**	■■■■ 20.3%
2	@ana	**14x**	■■■■ 20.3%
3	@maria	**14x**	■■■■ 20.3%
4	@devmaster	**14x**	■■■■ 20.3%
5	@techguru	**13x**	■■■ 18.8%



4. Total de URLs Encontradas

Métrica	Valor
Total de URLs detectadas	**80**
Total de posts analisados	**80**
Taxa URLs/post	**1.00**

Relatório gerado automaticamente · Análise de Redes Sociais · 14/04/2026

Temos o Código abaixo

```
import re
import string
import os
from collections import Counter
from itertools import combinations
import random

STOPWORDS = {
    "e","o","a","os","as","de","do","da","dos","das",
    "em","no","na","nos","nas","para","por","com",
    "um","uma","uns","umas","que","é","ao","à","se",
    "como","mais","mas","foi","ser","já","tem","há"
}

def gerar_posts():
    temas =
["python","dados","ia","tecnologia","programacao","web","machinelearni
ng"]
    usuarios = ["ana","joao","carlos","maria","devmaster","techguru"]
    frases = [
        "Aprendendo {} hoje",
        "Novo projeto sobre {}",
        "Estudando {} intensamente",
        "Adoro trabalhar com {}",
        "Explorando {} na prática"
    ]
    with open("posts.txt", "w", encoding="utf-8") as f:
        for _ in range(80):
            tema = random.choice(temas)
            usuario = random.choice(usuarios)
            frase = random.choice(frases).format(tema)
            post = f"{frase} #{tema} @{usuario}
```

```
https://site{random.randint(1,10)}.com\n"
            f.write(post)
    print(" Arquivo 'posts.txt' gerado com 80 posts.\n")

def ler_posts():
    with open("posts.txt", "r", encoding="utf-8") as f:
        for linha in f:
            yield linha.strip()  #  FIX: sem .lower() aqui, para
preservar URLs reais

def extrair_entidades(texto):
    #  FIX: re.IGNORECASE para capturar HTTPS:// e variações
    hashtags = re.findall(r"#\w+", texto.lower())
    mencoes  = re.findall(r"@\w+", texto.lower())
    urls     = re.findall(r"https?://\S+", texto, re.IGNORECASE)
    return hashtags, mencoes, urls

def limpar_texto(texto):
    texto = re.sub(r"https?://\S+", "", texto, flags=re.IGNORECASE)
    texto = re.sub(r"#\w+", "", texto)
    texto = re.sub(r"@\w+", "", texto)
    texto = texto.translate(str.maketrans("", "", string.punctuation))
    return texto.lower()

def analisar_padroes(hashtags_por_post):
    """Analisa co-ocorrência de hashtags nos posts."""
    pares = []
    for tags in hashtags_por_post:
        if len(tags) >= 2:
            pares.extend(combinations(sorted(set(tags)), 2))
    return Counter(pares)

def main():
    #  verificação correta sem resource leak
    if not os.path.exists("posts.txt"):
        gerar_posts()

    palavras        = []
    all_hashtags    = []
    all_mencoes     = []
    url_count       = 0
    hashtags_por_post = []  # para análise de padrões

    for post in ler_posts():
        hashtags, mencoes, urls = extrair_entidades(post)

        all_hashtags.extend(hashtags)
        all_mencoes.extend(mencoes)
        url_count += len(urls)
        hashtags_por_post.append(hashtags)

        texto_limpo = limpar_texto(post)
        tokens = texto_limpo.split()

        #  FIX: filtra stopwords E tokens não-alfabéticos (números,
etc.)
        palavras.extend([p for p in tokens if p not in STOPWORDS and
p.isalpha()])

    freq_palavras  = Counter(palavras)
    freq_hashtags  = Counter(all_hashtags)
```

```python
    freq_mencoes   = Counter(all_mencoes)
    freq_pares     = analisar_padroes(hashtags_por_post)

    # ─── FREQUÊNCIA ──────────────────────────────────────────────
    print("=" * 45)
    print("          ANÁLISE DE REDES SOCIAIS")
    print("=" * 45)

    print("\n Top 10 palavras mais frequentes:")
    print("-" * 35)
    for palavra, freq in freq_palavras.most_common(10):
        barra = "▇" * freq
        print(f"  {palavra:<18} {freq:>3}x  {barra}")

    print("\n  Top 5 hashtags:")
    print("-" * 35)
    for tag, freq in freq_hashtags.most_common(5):
        barra = "▇" * freq
        print(f"  {tag:<20} {freq:>3}x  {barra}")

    print("\n Top 5 menções:")
    print("-" * 35)
    for men, freq in freq_mencoes.most_common(5):
        barra = "▇" * freq
        print(f"  {men:<20} {freq:>3}x  {barra}")

    print(f"\n Total de URLs encontradas: {url_count}")

    # ─── PADRÕES ─────────────────────────────────────────────────
    print("\n Top 5 pares de hashtags que aparecem juntas (padrão):")
    print("-" * 45)
    if freq_pares:
        for (t1, t2), freq in freq_pares.most_common(5):
            print(f"  {t1} + {t2:<20} {freq:>3}x")
    else:
        print("  Nenhum par encontrado nos posts.")

    # ─── UTILIZADORES MAIS ACTIVOS ───────────────────────────────
    print("\n Utilizadores mais mencionados:")
    print("-" * 35)
    for men, freq in freq_mencoes.most_common(3):
        print(f"  {men:<20} {freq:>3} menções")

    print("\n" + "=" * 45)
    print(" Análise concluída!")
    print("=" * 45)

if __name__ == "__main__":
    main()
```